

学号： 2106410120



沈阳建筑大学

大数据技术基础 作业

2023 年秋季学期

学 院： 计算机科学与工程学院

专业班级： 计算机 2101 班

姓 名： 魏露林

第一章作业：

1.2

二、page 33

2.1 “大润发”、“沃尔玛”、“好德”和“农工商”四个超市都卖苹果、梨、香蕉、桔子和芒果五种水果。使用 NumPy 的 ndarray 实现以下功能。

- (1) 创建 2 个一维数组分别存储超市名称和水果名称；
- (2) 创建 1 个 4×5 的二维数组存储不同超市的水果价格，其中价格由 4 到 10 范围内的随机数生成；
- (3) 选择“大润发”的苹果和“好德”的香蕉，并将价格增加 1 元；
- (4) “农工商”水果大减价，所有水果价格减少 2 元；
- (5) 统计四个超市苹果和芒果的销售均价；
- (6) 找出桔子价格最贵的超市名称（不是编号）。

1. 程序代码

创建一个空的字符串列表

```
name_list = []
```

输入若干姓名，直到用户输入空行为止

```
while True:
```

```
    name = input("请输入姓名（或按 Enter 键结束输入）：")
```

```
    if name == "":
```

```
        break
```

```
    name_list.append(name)
```

输入要检索的姓名

```
search_name = input("请输入要检索的姓名：")
```

检查姓名是否存在于列表中

```
if search_name in name_list:
```

```
    print(f"{search_name} 存在于列表中。")
```

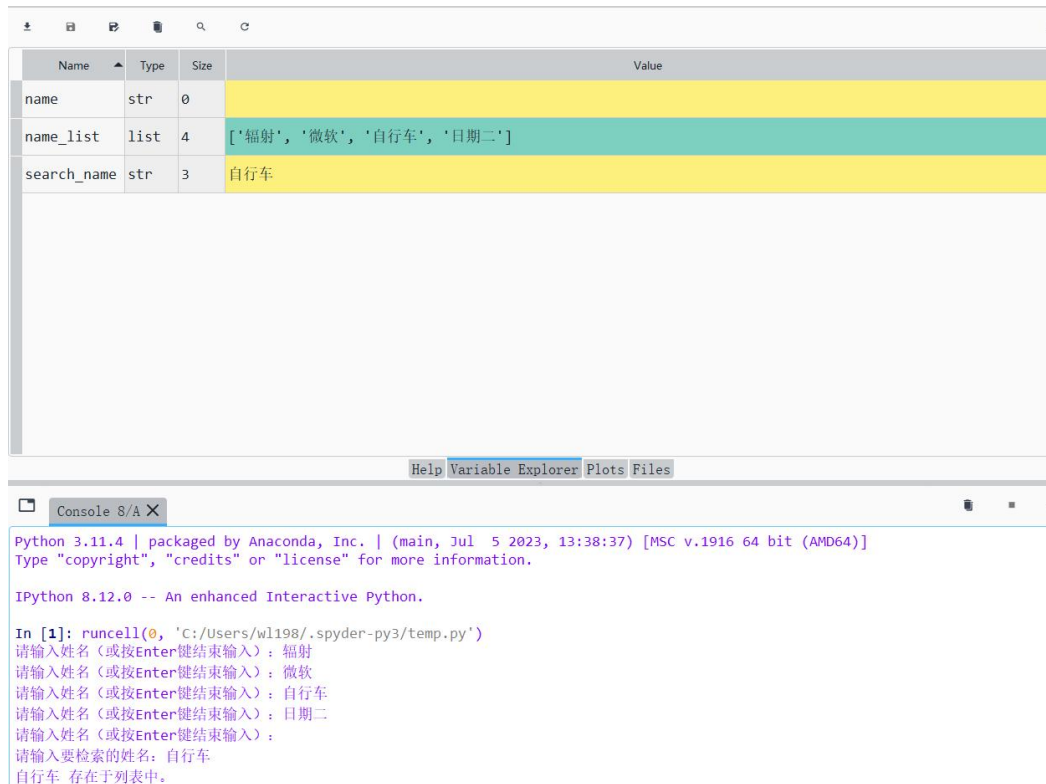
```
else:
```

```
    print(f"{search_name} 不存在于列表中。")
```

2. 程序说明

1. 创建一个空的字符串列表 `name\_list`，用于存储用户输入的姓名。
2. 使用一个无限循环，程序等待用户输入姓名，直到用户输入一个空行（按 Enter 键）为止。在每次迭代中，程序使用 `input` 函数来获取用户输入的姓名，并将这个姓名添加到 `name\_list` 列表中。
3. 用户输入的循环结束后，程序会要求用户输入要检索的姓名，并将输入存储在 `search\_name` 变量中。
4. 然后，程序使用 `in` 关键字来检查 `search\_name` 是否存在于 `name\_list` 中。如果存在，程序会打印出 "{search_name} 存在于列表中。"; 如果不存在，程序会打印出 "{search_name} 不存在于列表中。"

3. 程序截图



第二章作业：

page 33

2.1 “大润发”、“沃尔玛”、“好德”和“农工商”四个超市都卖苹果、梨、香蕉、桔子和芒果五种水果。使用 NumPy 的 ndarray 实现以下功能。

- (1) 创建 2 个一维数组分别存储超市名称和水果名称；
- (2) 创建 1 个 4×5 的二维数组存储不同超市的水果价格，其中价格由 4 到 10 范围内的随机数生成；
- (3) 选择“大润发”的苹果和“好德”的香蕉，并将价格增加 1 元；
- (4) “农工商”水果大减价，所有水果价格减少 2 元；
- (5) 统计四个超市苹果和芒果的销售均价；
- (6) 找出桔子价格最贵的超市名称（不是编号）。

1. 程序代码

```
import numpy as np
```

```
# (1) 创建 1 维数组存储超市名称和水果名称
```

```
supermarkets = np.array(["大润发", "沃尔玛", "好德", "农工商"])
```

```
fruits = np.array(["苹果", "梨", "香蕉", "桔子", "芒果"])
```

```
# (2) 创建 4x5 的二维数组存储不同超市的水果价格
```

```
prices = np.random.randint(4, 11, (4, 5))
```

```
# (3) 选择“大润发”的苹果和“好德”的香蕉，并将价格增加 1 元
```

```
prices[supermarkets=="大润发", fruits=="苹果"] = prices[supermarkets=="大润发", fruits=="苹果"] + 1
```

```
prices[supermarkets=="好德", fruits=="香蕉"] = prices[supermarkets=="好德", fruits=="香蕉"] + 1
```

```
# (4) “农工商” 水果大减价，所有水果价格减少 2 元
prices[supermarkets == "农工商"] -= 2
# (5) 统计四个超市苹果和芒果的销售均价
apple_prices = prices[:, fruits == "苹果"]
mango_prices = prices[:, fruits == "芒果"]
average_apple_price = np.mean(apple_prices)
average_mango_price = np.mean(mango_prices)
print(f"苹果的销售均价: {average_apple_price}")
print(f"芒果的销售均价: {average_mango_price}")
# (6) 找出桔子价格最贵的超市名称
max_orange_price_index = np.argmax(prices[:, fruits == "桔子"])
max_orange_price_supermarket = supermarkets[max_orange_price_index]
print(f"桔子价格最贵的超市是: {max_orange_price_supermarket}")
```

2. 程序说明

1. 创建两个一维 NumPy 数组 `supermarkets` 和 `fruits`，分别存储超市名称和水果名称。
2. 创建一个 4x5 的二维 NumPy 数组 `prices`，用于存储不同超市的水果价格，其中价格由 4 到 10 范围内的随机数生成。
3. 增加了"大润发"的苹果和"好德"的香蕉价格，分别增加了 1 元。
4. 减少了"农工商"超市的所有水果价格，每种水果减少 2 元。
5. 统计了四个超市的苹果和芒果的销售均价，并打印出结果。
6. 找出了价格最高的超市，即桔子价格最贵的超市，并打印出其名称。

3. 程序截图

Name	Type	Size	Value
apple_prices	Array of int32	(4, 1)	[[11] [5]
average_apple_price	float64	1	6.75
average_mango_price	float64	1	6.5
fruits	Array of str64	(5,)	ndarray object of numpy module
mango_prices	Array of int32	(4, 1)	[[6] [8]
max_orange_price_index	int64	1	1
max_orange_price_supermarket	str_	1	str_ object of numpy module
prices	Array of int32	(4, 5)	[[11 4 9 4 6] [5 4 5 7 8]
supermarkets	Array of str96	(4,)	ndarray object of numpy module

```
Python 3.11.4 | packaged by Anaconda, Inc. | (main, Jul 5 2023, 13:38:37) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 8.12.0 -- An enhanced Interactive Python.

In [1]: runcell(0, 'C:/Users/wl198/.spyder-py3/temp.py')
苹果的销售均价: 6.75
芒果的销售均价: 6.5
桔子价格最贵的超市是: 沃尔玛
```

第三章作业：

三、page 63 3.2

根据银行储户的基本信息，完成以下分析。

- 1) 从“bankpep.csv”文件中读取用户信息。
- 2) 查看储户的总数，以及居住在不同区域的储户数。
- 3) 计算不同性别储户收入的均值和方差。
- 4) 统计接受新业务的储户中各类性别、区域的人数。
- 5) 将存款账户、接受新业务的值转化为数值型。
- 6) 分析收入、存款账户与接收新业务之间的关系。

1. 程序代码

```
import pandas as pd
import numpy as np
# 1. 从“bankpep.csv”文件中读取用户信息
df = pd.read_csv('C:/Users/wl198/.spyder-py3/bankpep.csv')
# 2. 查看储户的总数，以及居住在不同区域的储户数
total_customers = len(df)
customers_by_region = df['region'].value_counts()
print(f"总储户数: {total_customers}")
print("居住在不同区域的储户数:")
print(customers_by_region)
# 3. 计算不同性别储户收入的均值和方差
mean_income_by_age = df.groupby('age')['income'].mean()
variance_income_by_gender = df.groupby('age')['income'].var()
# 4. 统计接受新业务的储户中各类性别、区域的人数
new_customers = df[df['pep'] == 'YES']
gender_region_counts = new_customers.groupby(['age', 'region']).size()
# 5. 将存款账户、接受新业务的值转化为数值型
df['save_act'] = df['save_act'].map({'YES': 1, 'NO': 0})
df['pep'] = df['pep'].map({'YES': 1, 'NO': 0})
# 6. 分析收入、存款账户与接收新业务之间的关系
correlation_income_savings = df['income'].corr(df['save_act'])
correlation_income_pep = df['income'].corr(df['pep'])
print(f"总储户数: {total_customers}")
print("居住在不同区域的储户数:")
print(customers_by_region)
print("\n 不同性别储户收入均值:")
print(mean_income_by_age)
print("\n 不同性别储户收入方差:")
print(variance_income_by_gender)
print("\n 接受新业务的储户中各类性别、区域的人数:")
print(gender_region_counts)
print(f"\n 收入与存款账户的相关系数: {correlation_income_savings}")
print(f"收入与接收新业务的相关系数: {correlation_income_pep}")
```

2. 程序说明

1. 导入库：

程序开始时导入所需的库，库包括 **Pandas** 和 **NumPy**。

2. 读取用户信息：

从名为“bankpep.csv”的 **CSV** 文件中读取用户信息

将数据加载到名为 **df** 的 **Pandas** 数据框中

3. 总储户数和按区域划分的储户数：

计算并打印以下信息：

数据集中的总客户数量。

不同区域中客户的数量统计。

4. 按年龄计算平均收入和按性别计算方差：

计算按年龄分组的平均收入以及按年龄分组的收入方差。

平均收入存储在 **mean_income_by_age** 变量中，方差存储在 **variance_income_by_gender** 变量中。

5. 接受新业务的客户统计：

识别已接受新业务机会的客户（其中“**pep**”为“**YES**”）。

然后按年龄和区域对这些客户进行计数和分组。

计数结果存储在 **gender_region_counts** 变量中。

6. 将分类值转换为数字：

将“**save_act**”和“**pep**”列中的分类值转换为数字值：

“**YES**” 被转换为 **1**。

“**NO**” 被转换为 **0**。

7. 分析关系：

计算收入与是否拥有储蓄账户之间的相关性，并将其存储在 **correlation_income_savings** 中。

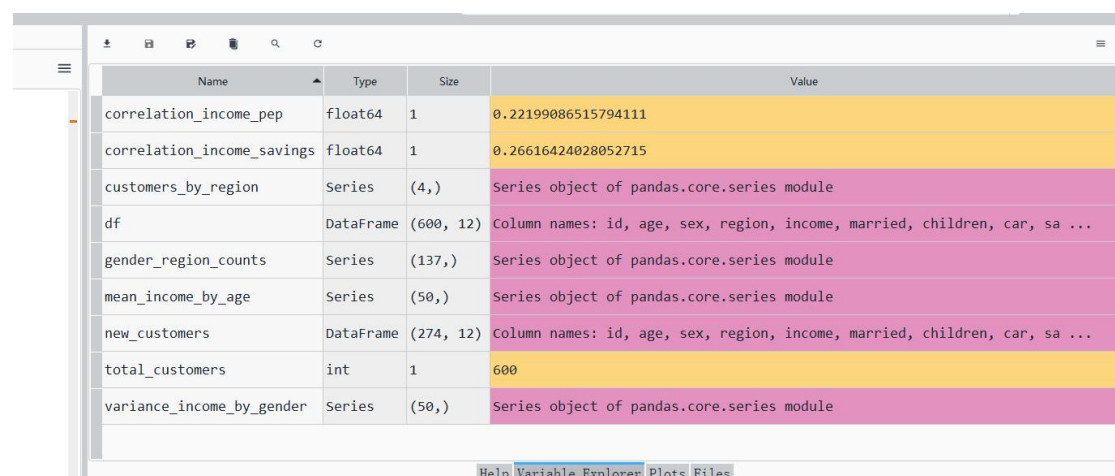
还计算了收入与是否接受新业务之间的相关性，并将其存储在 **correlation_income_pep** 中。

8. 打印结果：

程序打印分析结果，包括总客户数量、按区域划分的客户数量、按年龄计算的平均收入、按性别计算的收入方差、按性别和区域统计的接受新业务的客户数量，以及收入与储蓄账户、收入与是否接受新业务之间的相关性。

4. 程序截图

储存数据



Name	Type	Size	Value
correlation_income_pep	float64	1	0.22199086515794111
correlation_income_savings	float64	1	0.26616424028052715
customers_by_region	Series	(4,)	Series object of pandas.core.series module
df	DataFrame	(600, 12)	Column names: id, age, sex, region, income, married, children, car, sa ...
gender_region_counts	Series	(137,)	Series object of pandas.core.series module
mean_income_by_age	Series	(50,)	Series object of pandas.core.series module
new_customers	DataFrame	(274, 12)	Column names: id, age, sex, region, income, married, children, car, sa ...
total_customers	int	1	600
variance_income_by_gender	Series	(50,)	Series object of pandas.core.series module

收入均值

```
Python 3.11.4 | packaged by Anaconda, Inc. | (main, Jul 5 2023, 13:38:37) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 8.12.0 -- An enhanced Interactive Python.

In [1]: runcell(0, 'C:/Users/wl198/.spyder-py3/temp.py')
总储户数: 600
居住在不同区域的储户数:
INNER_CITY    269
TOWN          173
RURAL         96
SUBURBAN      62
Name: region, dtype: int64
总储户数: 600
居住在不同区域的储户数:
INNER_CITY    269
TOWN          173
RURAL         96
SUBURBAN      62
Name: region, dtype: int64

不同性别储户收入均值:
age
18    12105.694545
19    12971.085000
20    13808.870000
21    10865.977500
22    13184.491333
23    14102.594706
24    14487.463000
25    16798.054000
26    17730.510000
```

收入方差

```
Help Variable Explorer Plots Files

Console 4/A X

64    47242.423000
65    43106.500000
66    45238.220000
67    45515.414286
Name: income, dtype: float64

不同性别储户收入方差:
age
18    1.046322e+07
19    1.275805e+07
20    1.587230e+07
21    1.820974e+07
22    1.490680e+07
23    1.766083e+07
24    2.375283e+07
25    2.569233e+07
26    2.097160e+07
27    1.187366e+07
28    2.549279e+07
29    2.752460e+06
30    4.138806e+07
31    3.535816e+07
32    3.884839e+07
33    5.274515e+07
34    3.533056e+07
35    4.597524e+07
36    6.729321e+07
37    2.956255e+07
38    4.361452e+07
39    4.710547e+07
40    4.842167e+07
41    3.160651e+07
42    5.017624e+07

IPython Console History
```

收入相关分析

```
Console 4/A X
Name: income, dtype: float64

接受新业务的储户中各类性别、区域的人数:
age  region
18   INNER_CITY  1
    RURAL      1
19   INNER_CITY  1
    RURAL      2
    SUBURBAN   1
..
66   TOWN       3
67   INNER_CITY  3
    RURAL      1
    SUBURBAN   4
    TOWN       2
Length: 137, dtype: int64

收入与存款账户的相关系数: 0.26616424028052715
收入与接收新业务的相关系数: 0.22199086515794111

In [2]: aa
```

第四章作业:

二、page 86 可视化综合应用

文件 bankpep.csv 存放着银行储户的基本信息，请通过绘图对这些客户数据进行探索性分析。

- 1) 客户年龄分布的直方图和密度图
- 2) 客户年龄和收入关系的散点图
- 3) 绘制散点图观察账户（年龄，收入，孩子数）之间的关系，对角线显示直方图
- 4) 按区域展示平均收入的柱状图，并显示标准差
- 5) 多子图绘制：账户中性别占比饼图，有车的性别占比饼图，按孩子数的账户占比饼图
- 6) 各性别收入的箱须图

1. 程序代码

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
data = pd.read_csv('C:/Users/wl198/.spyder-py3/bankpep.csv')

data['age'].plot(kind = 'hist',bins = 10,title = 'Customer age distribution')
data['age'].plot(kind = 'kde',title = 'Customer age distribution',style = 'k-')

plt.xlabel('Age')
plt.ylabel('Density')
plt.show()

data[['age','income']].plot(kind = 'scatter',x = 'age',y = 'income',marker = 's',s
= 8,title = 'Customer Income',label = '(age,income)',grid = True,
```



```

xlim = [0, 80])      #s 设置点大小

plt.xlabel('Age')
plt.ylabel('Income')
plt.show()

pd.plotting.scatter_matrix(data[['age', 'income', 'children']], c = 'b')    #c 用来
设置颜色: m 为红紫色
plt.show()

mean = data.groupby(['region']).agg({'income': np.mean})
std = data.groupby(['region']).agg({'income': np.std})
mean.plot(kind = 'bar', yerr = std, rot = 45, title = 'Customer Income', legend =
False, color = 'r')
plt.xlabel('Region')
plt.show()

sex_data = data.groupby(['sex'])['sex'].count()
car_data = data[data['car'] == 'YES'].groupby(['sex'])['sex'].count()
children_data = data.groupby(['children'])['children'].count()
fig = plt.figure(figsize = (7, 6))
fig.add_subplot(2, 2, 1)
sex_data.plot(kind = 'pie', title = 'Customer Sex', startangle = 60, autopct =
'%1.1f%%')
fig.add_subplot(2, 2, 2)
car_data.plot(kind = 'pie', title = 'Customer Car Sex', startangle = 60, autopct =
'%1.1f%%')
fig.add_subplot(2, 2, 3)
children_data.plot(kind = 'pie', title = 'Customer Children', startangle =
60, autopct = '%1.1f%%')
plt.savefig('饼图.jpg', dpi = 400, bbox_inches = 'tight')
plt.show()

data[['income', 'sex']].boxplot(by = 'sex', figsize = (6, 6))
plt.show()

```

2. 程序说明

1. 导入库

2. 读取数据

3. 绘制年龄分布直方图和核密度估计图:

使用`data['age']`列的数据绘制了年龄分布的直方图。

使用`plt.show()`显示图形。

4. 绘制年龄和收入的散点图:

使用`data[['age', 'income']]`列的数据绘制了年龄和收入的散点图。

使用`plt.show()`显示图形。

5. 绘制散点矩阵图:

使用`pd.plotting.scatter\_matrix`函数绘制了年龄、收入和子女数量之间的散点矩阵图。

使用`plt.show()`显示图形。

6. 绘制不同区域客户平均收入柱状图:

使用`data.groupby(['region']).agg({'income': np.mean})`计算了不同区域客户的平均收入。

使用`data.groupby(['region']).agg({'income': np.std})`计算了不同区域客户的收入标准差。

绘制了柱状图，其中包括误差条，展示了不同区域客户的平均收入。

7. 绘制性别、是否有车和子女数量的饼图:

使用`data.groupby(['sex'])['sex'].count()`统计了不同性别客户的数量，然后绘制了性别分布的饼图。

使用`data[data['car'] == 'YES'].groupby(['sex'])['sex'].count()`统计了拥有车辆的不同性别客户的数量，然后绘制了拥有车辆的客户性别分布的饼图。

使用`data.groupby(['children'])['children'].count()`统计了不同子女数量的客户数量，然后绘制了子女数量分布的饼图。

使用`plt.savefig()`保存饼图为图像文件。

8. 绘制性别和收入的箱线图:

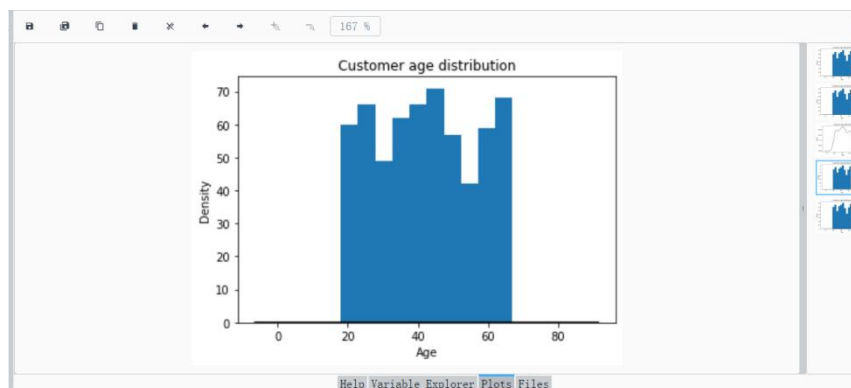
使用`data[['income', 'sex']].boxplot(by='sex', figsize=(6,6))`绘制了性别和收入的箱线图,用于可视化不同性别客户的收入分布。

使用`plt.show()`显示图形。

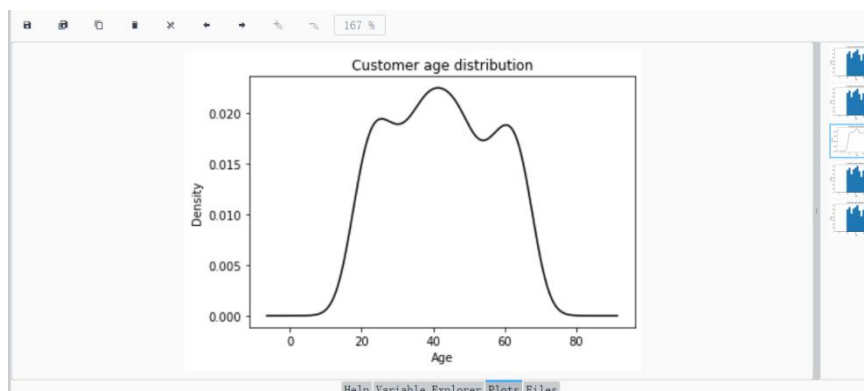
3. 程序截图

1) 客户年龄分布的直方图和密度图

直方图



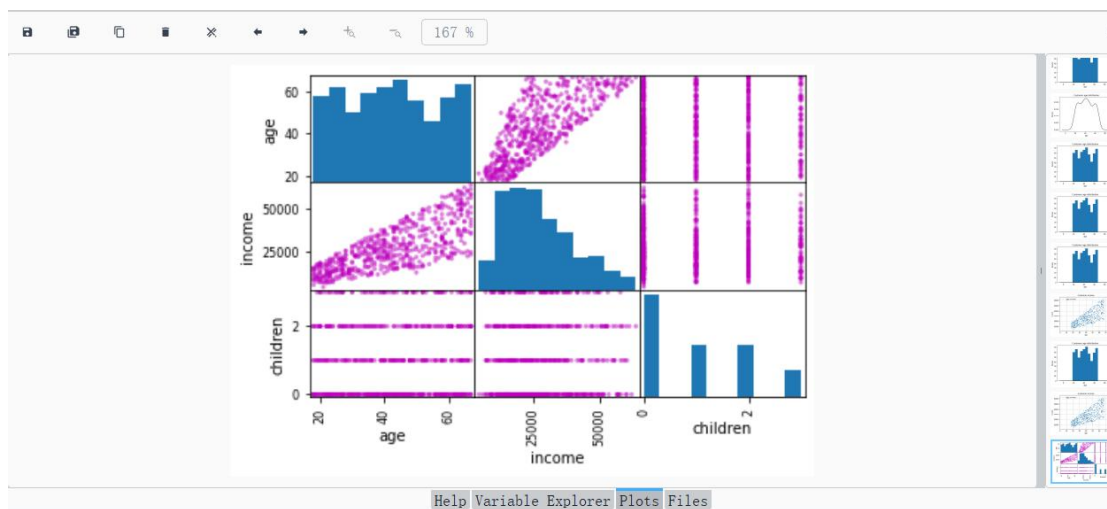
密度图



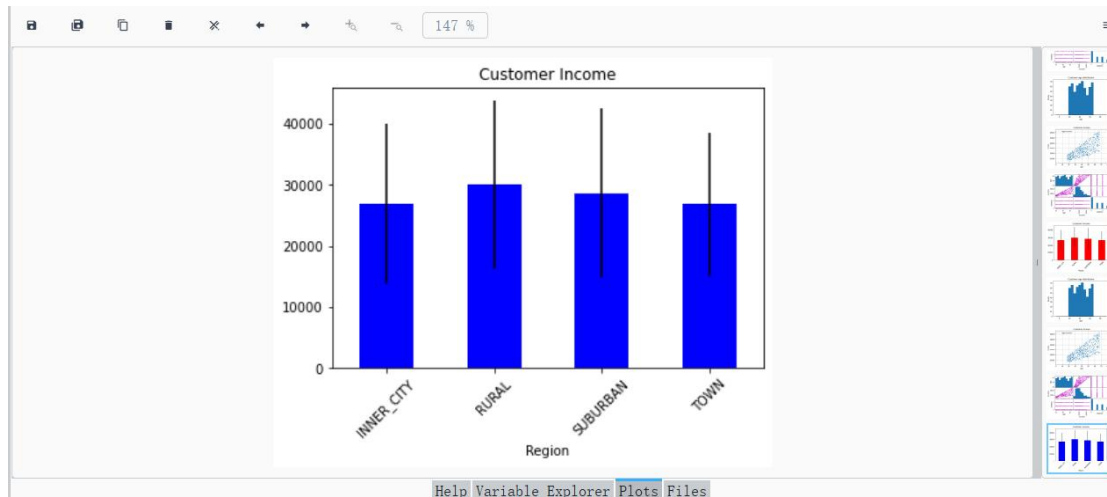
2) 客户年龄和收入关系的散点图



3) 绘制散点图观察账户（年龄，收入，孩子数）之间的关系，对角线显示直方图



4) 按区域展示平均收入的柱状图，并显示标准差



5) 多子图绘制：账户中性别占比饼图，有车的性别占比饼图，按孩子数的账户占比饼图



6) 各性别收入的箱须图

